Verification and Validation Report: Software Engineering

Team 13, Speech Buddies
Mazen Youssef
Rawan Mahdi
Luna Aljammal
Kelvin Yu

March 9, 2026

1 Revision History

Date	Version	Notes
March 8th, 2026	1.0	Added Non-Functional Performance tests
March 9th, 2026	1.0	Added Non-Functional Usability and Humanity tests
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

symbol	description
T	Test

[symbols, abbreviations or acronyms – you can reference the SRS tables if needed —SS]

Contents

List of Tables

List of Figures

This document ...

3 Functional Requirements Evaluation

This section presents the evaluation results for all functional requirements as defined in the SRS. Each functional requirement is evaluated through the corresponding test cases from Section 4.1 of the V&V Plan. For detailed test specifications (initial state, inputs, expected outputs, and execution methodology), refer to the V&V Plan. This section focuses on test results and traceability.

3.1 FR1: Accept Speech Audio via Microphone

Requirement: The system must capture live speech input from the user through a standard built-in or external microphone, reliably detecting and recording audio from default OS microphone devices across Windows, macOS, iOS, and Android, with a minimum 16 kHz sampling rate.

3.1.1 test-FR1-MIC-1

Summary: Verify microphone permission request and audio capture.
Result: Pass – microphone permissions were requested as expected on Windows, and audio was successfully captured at 16 kHz or higher across all test runs.

3.1.2 test-FR1-MIC-2

Summary: Verify permission handling on iOS and Android platforms.
Result: Pass – permission handling was reliable on both iOS and Android, and microphone access was consistently verified through each platform's device permissions toolbar.

3.1.3 test-FR1-MIC-3

Summary: Verify automatic fallback when audio devices are disconnected during capture.
Result: Pass – when the active audio device was disconnected, the system detected the change and automatically switched to the default device.

3.2 FR2: Convert Impaired Speech to Text with ≥80% Accuracy

Requirement: The system must process captured audio and output textual representation. Transcription accuracy must reach at least 80% WER reduction compared to baseline models, and achieve ≥80% accuracy for common commands.

3.2.1 test-FR2-ASR-1: Dataset Evaluation vs Baseline

Summary: Automated batch evaluation comparing system WER to baseline on impaired-speech test set.
Result: Pass – the system achieved a WER reduction greater than 80% relative to the baseline across the impaired-speech datasets

3.2.2 test-FR2-ASR-2: Common Command Recognition

Summary: Automated evaluation of transcription accuracy for 50+ common command intents.
Result: Pass – transcription accuracy for the 50+ common command intents exceeded 80%, with most commands recognized correctly on the first attempt.

3.3 FR3: Display Transcription for Verification

Requirement: The transcribed text must be displayed in real time on the user's device interface. Every spoken input is displayed within 2 seconds as text on the UI, with at least 95% consistency across trials.

3.3.1 test-FR3-UI-1: Real-Time Display Latency

Summary: Automated measurement of time-to-first-text for 100 utterances across platforms.
Result: Partial – some utterances were displayed within approximately 2 seconds, but most trials experienced additional latency of a couple of seconds before the first text appeared.

3.3.2 test-FR3-UI-2: Continuous Update & Finalization

Summary: Manual verification of incremental text updates and finalization cues during streaming transcription.
Result: Partial – continuous streaming transcription is not yet implemented, but text updates do appear on the UI once recognition completes, so only finalization behavior could be partially observed.

3.4 FR4: Map Text to Arbitrary Device Commands

Requirement: The system must recognize when transcribed text corresponds to a predefined device action and translate it into the appropriate command representation.

3.4.1 test-FR4-MAP-1: Canonical Command Set Accuracy

Summary: Automated evaluation of mapping accuracy for 50 predefined commands.
Result: Pass – commands such as "open Chrome", and "play video" were consistently mapped to the correct internal device actions with high accuracy.

3.4.2 test-FR4-MAP-2: Paraphrase & Synonym Robustness

Summary: Automated evaluation of mapping accuracy with paraphrased and synonymous command phrasings.
Result: Pass – paraphrased commands like "launch Chrome" or "go down the page" were correctly interpreted as their canonical equivalents (e.g., "open Chrome", "scroll down") in the vast majority of cases.

3.4.3 test-FR4-MAP-3: Ambiguity & Disambiguation

Summary: Manual verification of clarification prompts for ambiguous commands.
Result: Pass – when users issued ambiguous commands such as "Try YouTube", the system responded with clarification prompts (e.g., say yes to confirm opening YouTube) and executed the chosen option correctly.

3.4.4 test-FR4-MAP-4: Out-of-Domain Commands

Summary: Manual verification of feedback for unsupported commands.
Result: Pass – for unsupported or out-of-domain requests like "Hack this Website", the system did not attempt execution and instead returned clear feedback that the command was not supported.

3.5 FR5: Execute Commands on the Host Device

Requirement: The system must execute mapped commands through the host device's accessibility framework or APIs, resulting in visible user action.

3.5.1 test-FR5-EXEC-1: End-to-End Execution Latency & Reliability

Summary: Automated measurement of execution latency and success rate for 50-command set across platforms.
Result: Pass – end-to-end command execution completed successfully in roughly 5–10 seconds for most commands, with an overall success rate of about 90%.

3.5.2 test-FR5-EXEC-2: Accessibility/API Fallback Paths

Summary: Verification of fallback behavior when primary API is disabled.
Result: Partial – when the primary API was disabled, commands were not able to be executed and robust fallback mechanisms still need to be implemented.

3.5.3 test-FR5-EXEC-3: Safety & Confirmation for Destructive Actions

Summary: Manual verification of confirmation prompts for destructive commands.
Result: Pass – for destructive prompts such as deleting files or modifying the website, built-in guardrails prevented immediate execution and required explicit confirmation, effectively blocking unsafe actions.

3.6 FR6: Handle Security Software Interference

Requirement: The system must detect when browser, operating system, or antivirus security controls block or restrict automated actions and provide clear feedback to the user, including the affected command and available fallback options. No specific test cases were defined in the V&V Plan for FR6. This requirement was evaluated through scenario-based testing during command execution tests FR5.

4 Nonfunctional Requirements Evaluation

This section evaluates the selected nonfunctional requirements defined in the VnV Plan. The assessment focuses on two key quality attributes: performance and usability/humanity. The following subsections describe the tests conducted and the results obtained for each category.

4.1 Performance

This subsection evaluates the system's performance under typical operating conditions. The tests focus on measuring response time and overall system responsiveness to ensure that the application operates efficiently during normal use.

4.2 test-PF-1: ASR Latency Benchmark

Type Automated, Dynamic

Test Case Derivation PRF-13.1

Test Summary The ASR latency benchmark was executed using 20 audio samples spoken by various speakers. For each sample, latency was measured from the marked end-of-speech point to the time at which the transcript became available. The measured latencies were recorded and averaged to evaluate performance against the requirement threshold.

Table 1: ASR latency benchmark results for 20 audio samples

#	**Sample Text**	**Length (words)**	**Latency (s)**
1	open google	2	5.81
2	google.com	1	6.30
3	search Wikipedia about paris	4	6.08
4	navigate to paris's sport section	5	5.93
5	open gmail	2	4.73
6	open google and search for keyboards	6	5.03
7	go on bestbuy and search for newest game	8	5.43
8	search software engineering jobs	4	4.48
9	search how to boil pasta	5	5.21
10	search used honda civic	4	6.08
11	search toronto raptors schedule	4	6.50
12	go to the history section of the canada page	9	7.01
13	search apple stock price	4	4.81
14	search directions to union station	5	5.21
15	open reddit	2	6.00
16	play jazz music on youtube	5	6.03
17	search restaurants near me	4	5.95
18	open wikipedia	2	4.83
19	search mcmaster university	3	6.80
20	open ebay	2	5.00
		Average Latency	**5.66**

Test Data and Results

Calculation of Average Latency The average latency is calculated as:

$$\text{Average Latency} = \frac{\sum_{i=1}^{20} L_i}{20} = 5.66 \text{ s}$$

where L_i is the latency measured for sample i.

Actual Result The measured average latency across the 20 audio samples was 5.66 seconds.

Conclusion This test **does not fully satisfy** PRF-13.1, since the measured average latency exceeded the required threshold of 5 seconds.

4.2.1 test-PF-2: Command Pipeline Latency

Type Automated, Dynamic

Test Case Derivation PRF-13.2

Test Summary The command pipeline latency benchmark was run using the same 20 audio samples from the ASR latency test. For each sample, latency was measured from the point at which the transcript became available to the point at which the requested command result was completed by the system. The measured latencies were recorded and averaged to assess compliance with the performance requirement.

Table 2: Command pipeline latency results for 20 audio samples

#	**Sample Text**	**Length (words)**	**Latency (s)**
1	open google	2	8.92
2	google.com	1	9.41
3	search Wikipedia about paris	4	12.87
4	navigate to paris's sport section	5	14.35
5	open gmail	2	9.76
6	open google and search for keyboards	6	13.42
7	go on bestbuy and search for newest game	8	15.91
8	search software engineering jobs	4	12.26
9	search how to boil pasta	5	11.83
10	search used honda civic	4	13.74
11	search toronto raptors schedule	4	12.95
12	go to the history section of the canada page	9	18.62
13	search apple stock price	4	10.54
14	search directions to union station	5	14.08
15	open reddit	2	9.88
16	play jazz music on youtube	5	13.67
17	search restaurants near me	4	12.43
18	open wikipedia	2	9.15
19	search mcmaster university	3	11.96
20	open ebay	2	8.64
		Average Latency	**12.52**

Test Data and Results

Calculation of Average Latency

$$\begin{aligned}\text{Average Latency} &= \frac{1}{20}(8.92 + 9.41 + 12.87 + 14.35 + 9.76 + 13.42 + 15.91 + 12.26 + 11.83 + 13.74 \\ &\quad + 12.95 + 18.62 + 10.54 + 14.08 + 9.88 + 13.67 + 12.43 + 9.15 + 11.96 + 8.64) \\ &= 12.52 \text{ s}\end{aligned}$$

Actual Result The measured average command pipeline latency across the 20 audio samples was 12.52 seconds.

Conclusion This test satisfied the performance requirement, since the measured average latency was below the 15 second threshold.

4.2.2 test-PF-3: Accuracy & Precision (Stationary Noise)

Type Offline Evaluation

Test Case Derivation PRF-13.1, PRF-13.2

Test Summary This test was planned to evaluate ASR and command classification performance under stationary background noise conditions, including AC and fan noise mixed at 20 dB, 10 dB, and 5 dB SNR. The intended evaluation metrics were word recognition accuracy and command precision under noisy conditions.

Result This test was not executed during the current development cycle.

Rationale Robustness to stationary background noise was identified as a stretch goal for the project. At this stage, development and validation efforts were prioritized toward core functionality under standard operating conditions. As a result, noise robustness testing was deferred and is outside the current project scope.

Conclusion No result is reported for this test. Evaluation of performance under stationary noise remains future work.

4.3 Usability and Humanity Requirements Evaluation

Testing was conducted with two external participants Prerna and Sarah. The microphone was active throughout each session in a quiet room with no other speakers present. Each participant used a different confirmation UI mode: Prerna used the *hidden* confirmation style (yes/no voice response) and Sarah used the *visible* confirmation style (clickable Confirm/Cancel buttons).

4.3.1 test-UH-1: Core Task Ease of Use

Type User Testing, Instrumented

Test Case Derivation UHR-12.1

Fit Criterion Both participants complete their tasks in $\leq$4 interactions; success feedback observed within 1 second of action.

Test Summary Each participant completed three core tasks: speaking a command, observing the transcription, and confirming the system's interpreted response, using their assigned confirmation style. After completing the tasks each participant rated overall ease of use on a 1–5 scale and provided brief comments.

Table 3: Core task completion and ease-of-use ratings ($n = 2$). Scale: 1 = very difficult, 5 = very easy.

User	Confirm Style	Tasks Completed	Rating (1–5)	Comments
Prerna	Spoken	3/3: spoke command, observed transcription, confirmed via voice	4/5	Found flow natural; initially unsure when to speak
Sarah	Buttons	3/3: spoke command, observed transcription, confirmed via button	5/5	Buttons made confirmation step very clear
	Summary	6/6 tasks (100%)	**4.5/5 avg**	Both styles usable

Test Data and Results After the user confirmed the command, the system initiated execution of the corresponding operation within approximately

0.78 seconds (0.68 seconds for Sarah, 0.87 seconds for Prerna) across all trials, satisfying the 1-second responsiveness requirement. During testing, users suggested that displaying a visible completion message (e.g., "Completed") would improve feedback by clearly indicating when the command has finished executing.

Analysis Both participants completed all three tasks successfully on their first attempt across both confirmation styles. The average ease-of-use rating of 4.5/5 indicates the core workflow is intuitive. Prerna's rating of 4 reflected a brief moment of uncertainty about when to speak—a discoverability gap in the hidden confirmation style rather than a structural usability issue. Sarah's rating of 5 confirms that the visible button style removes this ambiguity. Both styles produced successful outcomes, validating the system's support for multiple interaction preferences.

Conclusion This test **satisfies** UHR-12.1. All tasks were completed within the interaction threshold, feedback timing was met, and the average ease-of-use rating of 4.5/5 exceeds the acceptable threshold.

4.3.2 test-UH-2: Onboarding Learnability & First-Try Success

Type User Testing, Survey

Test Case Derivation UHR-12.3

Fit Criterion $\geq$85% complete all core tasks on first attempt; onboarding time $\leq$10 minutes on average; satisfaction rated out of 5 with open-ended feedback on liked/disliked aspects.

Context VoiceBridge does not currently include a dedicated onboarding tutorial. The UI itself provides instructional cues with labelled controls, status messages, and contextual prompts to guide first-time users.

Test Summary Participants were given no briefing beyond a general description of VoiceBridge's purpose. They completed the full core interaction

sequence without any guidance. Task time was recorded from first interaction to final confirmation. Participants then rated their experience on a 1–5 scale and provided open-ended feedback.

Table 4: First-try success, onboarding time, ratings, and feedback ($n = 2$). Scale: 1 = very poor, 5 = excellent.

User	**1st-Try**	**Time**	**Rating (1–5)**	**Feedback**
Prerna	3/3	4m 22s	4/5	Liked: clear step-by-step flow. Disliked: unclear if command was executed successfully
Sarah	3/3	3m 48s	5/5	Liked: buttons intuitive and obvious. Disliked: Slight uncertainty about when command is complete
Summary		<5 min avg	**4.5/5 avg**	100% first-try success; both under 10-minute target

Test Data and Results

Analysis Both participants achieved 100% first-try success with no assistance, and both completed the sequence well under the 10-minute threshold. The primary criticism—initial uncertainty about when to speak—is specific to the hidden confirmation style and points to a UI signal gap rather than a broader learnability problem. Sarah experienced no such confusion with the visible button style.

Conclusion This test **satisfies** UHR-12.3. Both participants achieved first-try success under the time threshold. The listening-readiness signal gap is logged in Section 7.

4.3.3 test-UH-3: Personalization Ability

Type Manual, Dynamic

Test Case Derivation UHR-12.2

Fit Criterion Personalized interaction should be consistent and predictable; user preferences must persist without unexpected resets.

Test Summary This evaluation assessed the **consistency, predictability, and persistence** of personalization mechanisms, including language/confirmation style and UI preferences (e.g., additional action buttons). The system was tested across session resets to confirm that preferences are retained and applied consistently.

Table 5: Personalization NFR observations.

Preference	**Observed Consistency/Behavior**	**Result**
Language / Confirmation Style	Settings consistently applied; interaction behavior matched user expectations across sessions	Meets NFR
UI preference — action buttons	Enabled buttons remained visible and accessible; consistent behavior after session reset	Meets NFR
Cross-session persistence	Preferences persisted across browser restarts and re-authentication	Meets NFR

Test Data and Results

Analysis All personalization mechanisms operated predictably and consistently. Preferences were successfully retained across sessions, satisfying the NFR goals for consistency, predictability, and persistence.

Conclusion This test **satisfies** UHR-12.2. Personalization behavior meets NFR expectations, with consistent interaction and cross-session preference retention.

4.3.4 test-UH-4: Message Clarity & Politeness

Type Survey-Based Language Review

Test Case Derivation UHR-12.4

Fit Criterion ≥90% of participants rate system messages ≥4/5 for clarity and tone.

Test Summary After completing their task sessions, both participants rated eight representative VoiceBridge system messages on two separate 1–5 Likert scales: *clarity* (1 = very unclear, 5 = very clear) and *tone* (1 = impolite or unhelpful, 5 = appropriate and supportive). Messages covered the full range of interaction states.

Table 6: Per-message clarity and tone ratings ($n = 2$). Scale: 1–5.

Message / Scenario	**P Clar**	**A Clar**	**P Tone**	**A Tone**	**Avg**
"Which email service would you like to open? Gmail, Outlook, Yahoo Mail, or something else?" + "Maps to Gmail..."	–	5	–	5	5.0
"Play videos about McMaster University on YouTube"	5	–	5	–	5.0
"I'm not sure I understand. Could you rephrase that?"	2	–	4	–	3.0
"Listening... Speak your command"	5	4	5	5	4.8

Test Data and Results Of the individual clarity ratings, 80% were ≥4/5, which is below the 90% target. For tone ratings, 100% were ≥4/5, exceeding the target. P = Prerna, S = Sarah.

Analysis Overall, message tone is consistently strong. However, the "I'm not sure I understand. Could you rephrase that?" message received a low clarity score, highlighting it as the primary area for improvement. This message indicates a problem but does not provide a concrete next step or specify which part of the input was unclear.

Conclusion This test **partially satisfies** UHR-12.4. The primary issue is the "I'm not sure I understand. Could you rephrase that?" message, which has been identified for rewording in future iterations to improve clarity and user guidance.

4.3.5 test-UH-5: Assistive Navigation

Type Manual Keyboard Testing

Test Case Derivation UHR-12.5

Fit Criterion All interactive elements reachable and operable via keyboard.

Test Summary Keyboard-only navigation was performed following WCAG accessibility principles, with no formal automated audit conducted at this stage. Keyboard-only navigation was performed on all currently accessible interactive elements:

- Light/Dark mode toggle
- Visible confirmation toggle

Each element was successfully reached using the Tab key and activated with Enter or Space. Visual focus was clearly indicated when navigating with the keyboard. No automated accessibility audit was performed, but these core controls function correctly and comply with WCAG keyboard accessibility principles.

Analysis Within the tested scope, the interface supports predictable and consistent keyboard navigation. As additional interactive elements are added in future revisions, similar verification should be conducted to ensure ongoing accessibility compliance.

Conclusion This test **satisfies** the current scope of UHR-12.5. Keyboard accessibility is confirmed for all present interactive elements, and the system adheres to expected nonfunctional accessibility requirements.

5 Comparison to Existing Implementation

This section will not be appropriate for every project.

6 Unit Testing

Unit testing for *VoiceBridge* focused on the core backend modules defined in the MIS and prioritised in the V&V Plan: the Speech-to-Text Engine (M4), Command Orchestrator (M5), Browser Orchestrator (M7), Session Manager (M8), Real-time Audio Capture (M15), and the Feedback Module (M20). For each of these modules we implemented automated tests using `pytest` that cover normal behaviour for all public access routines, representative edge cases, and key error paths (for example, invalid configuration, unavailable external services, malformed inputs, and I/O failures). Heavy external dependencies such as cloud LLM APIs, browser automation libraries, audio hardware, ASR model checkpoints, and the filesystem are replaced with test doubles where appropriate, allowing the unit tests to run deterministically and without network or device access. As a result, the unit test suite can be run quickly and repeatedly during development and in CI, and it provides direct evidence that the main control and data-flow responsibilities of each module behave as specified in the MIS.

The following figure captures the automatically generated result which summarize the results of the unit tests.

```
$ pytest
========================================================== test session starts ==========================================================
platform win32 -- Python 3.13.12, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\Rawan Alamily\Downloads\capstone\voicebridge
plugins: anyio-4.12.1, asyncio-1.3.0, cov-7.0.0, html-4.2.0, metadata-3.1.1
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 274 items

test\test_app\test_command_orchestrator.py ..............................................................................     [ 28%]
test\test_app\test_stt.py ...................................................                                                [ 46%]
test\test_control\test_browser_controller.py ............                                                                   [ 51%]
test\test_control\test_session_manager.py ......................                                                            [ 59%]
test\test_data\test_feedback_module.py ......................................                                               [ 74%]
test\test_inputs\test_audio_capture.py ......................................................................               [100%]

========================================================== 274 passed in 1.44s ==========================================================
```

Figure 1: Terminal report of unit test result by module test. The bottom line summarizes the number of tests passed.

7 Changes Due to Testing

[This section should highlight how feedback from the users and from the supervisor (when one exists) shaped the final product. In particular the feedback from the Rev 0 demo to the supervisor (or to potential users) should be highlighted. —SS]

8 Automated Testing

Automated tests are executed locally via `pytest` and in continuous integration using GitHub Actions. The CI pipeline runs the full Python unit test suite on every push and pull request, including tests under `test/test_app`, `test/test_control`, and `test/test_inputs`, and fails the build if any test fails or if the test run crashes due to missing dependencies. Where necessary, platform-specific tests (for example, those that depend on Windows event loop policies) are guarded with `skip` conditions so that the suite remains portable across Linux-based CI runners and local Windows development machines. In addition to unit tests, the pipeline runs static analysis and style tools (such as `ruff` and `eslint`/`prettier` in the frontend) to catch common errors and enforce consistency before code is merged.

9 Trace to Requirements

10 Trace to Modules

Table **??** summarises how the implemented unit test suites map back to the MIS modules and, indirectly, to the functional requirements they support. This table is intentionally aligned with the Unit Test Description section of the V&V Plan; in this report we only record modules for which unit tests were actually implemented.

Table 7: Traceability Between Unit Tests and MIS Modules

MIS Module	**Primary Responsibilities**	**Unit Test File(s)**
M4 Speech-to-Text Engine	Audio format validation, ASR model integration, transcription pipeline, personalization	`test/test_app/test_stt.py`
M5 Command Orchestrator	LLM prompt construction, response parsing, clarification handling, safety guardrails	`test/test_app/test_command_orchestrator.py`
M7 Browser Orchestrator	Delegating textual tasks to browser automation agent	`test/test_control/test_browser_controller.py`
M8 Session Manager	Lifecycle of shared browser session (start/stop/get)	`test/test_control/test_session_manager.py`
M15 Audio Capture Module	Microphone capture, VAD-based speech detection, utterance segmentation, audio statistics	`test/test_inputs/test_audio_capture.py`
M20 Feedback Module	Persistent storage of structured command feedback records	`test/test_data/test_feedback_module.py`

Modules in the presentation layer and any auxiliary utilities that are not listed in Table **??** were not unit-tested during this iteration, either because they are thin UI shells around backend APIs or because they were judged lower priority for the current MVP. Their behaviour is instead validated

through end-to-end system tests, manual exploratory testing, and usability evaluations described earlier in this document.

11 Code Coverage Metrics

Code coverage for the Python backend was measured using `pytest-cov`, which instruments the code during unit test runs and produces line- and branch-coverage reports. The following figures capture the final state of coverage at the end of the project. The primary goal was to achieve high coverage for the safety- and control-critical modules (M4–M5 and M7–M8) and reasonable coverage for supporting components such as M15, rather than maximising overall project-wide coverage.

```
_____________________________________________ coverage: platform win32, python 3.13.12-final-0
===============…============

Name                                    Stmts   Miss  Cover
-------------------------------------------------------------
src\app\command_orchestrator.py           116     11    91%
src\app\speech_to_text_engine.py           95     16    83%
src\control\browser_orchestrator.py        16      0   100%
src\control\session_manager.py             16      0   100%
src\data\feedback_store.py                 24      0   100%
src\input\audio_capture.py                173      0   100%
-------------------------------------------------------------
TOTAL                                     440     27    94%
```

Figure 2: Terminal report of unit test coverage by module. Note that only modules deemed in scope, as mentioned in the VnV Plan are included

References

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)